KORE-OS - Complete Project Documentation

Version: 1.0 (recursive build, section by section)

Status: In progress - Sections 1-2 complete

Source: HERMES-KORE-SPECIFICATION.md (lines 1-214), Kore-OS_TEP implementation, Hermes-Kore fusion study

Commanditaire: K Ray Nobun (The_Korp)

Table of Contents (Progress)

- Context and Objective - COMPLETE
- Separation of Communication vs Reflection - COMPLETE
- Automatic API Model Switching - COMPLETE
- Dedicated Skills Directory and Traceability - COMPLETE
- Multiple Sessions and Alternation - COMPLETE
- Real-Time VPS Monitoring - COMPLETE
- Dynamic Skills Management - COMPLETE
- Sub-Agent Roles - COMPLETE
- Lanes (Parallel Work Tracks) - COMPLETE
- Dashboard Workers and Purpose Narration - COMPLETE
- Provider Quotas and Color Coding by Model - COMPLETE
- 100% Automatic Background Tasks - COMPLETE
- Instructions View and Status by Lane - COMPLETE
- Delivery Modalities for Worker "Pythagor" - COMPLETE
- The Worker Class Model (Revolutionary Concept) - COMPLETE
- Skills System: Two-Tier Architecture - COMPLETE
- Prompts, Identity, and AI Models - COMPLETE
- Internal Logic and Workflows - COMPLETE
- Communication and Notification Chain - COMPLETE
- Reliability, Recovery, and Limits - COMPLETE
- Integration with Hermes and Kore-OS - COMPLETE

1. Context and Objective

1.1 What currently exists

Hermes is an AI agent that runs primarily through a messaging interface - Telegram. You (K Ray Nobun, also known as The_Korp) interact with Hermes by sending messages in a Telegram chat. Hermes receives your instructions, carries out work using its available tools and skills, and sends responses back in the same chat.

Hermes is not just a chatbot. It is an orchestrator that can:

- Read and write files on your VPS

- Run terminal commands

- Search and extract content from the web

- Navigate websites through a browser

- Manage connections to external services (hosted connectors and MCP servers)

- Manage skills (reusable procedures stored in directories)

- Maintain persistent memory (a profile about you and a store of learned notes)

- Synthesize speech (text-to-speech)

- Analyze images

Hermes delegates work to sub-agents (workers) that run for limited periods. It uses AI models from different providers (NVIDIA, OpenRouter, Anthropic, etc.) with automatic fallback when one fails. It runs on a VPS (Linux, 2 CPU cores, ~7.4 GB RAM, swap saturated at 1023 MB, disk 39% used).

1.2 What is not working well today

You experience several frictions with the current Hermes setup:

-
Communication and reflection are mixed. Important results - answers to your instructions, deliverables, validations, blocking errors - appear in the same chat stream as Hermes's internal reasoning, self-correction, and debug logs. You cannot easily tell which messages are meant for you and which are internal thinking.

-
No visual dashboard. There is no screen where you can see at a glance what sub-agents are working on, what is finished, what is blocked, or what the infrastructure looks like.

-
No real-time infrastructure monitoring. Your VPS resources (RAM, CPU, SWAP, disk) are not shown live. You only get snapshots from cron deliveries, not a continuous view.

-
Tasks of background are not fully automatic. When a background task runs, the system may ask afterward whether it should resume, which breaks automation.

-
No worker roles are visible. Sub-agents do not have clearly defined or displayed roles (admin, supervisor, worker, cronjobber, researcher).

-
Skills are not visible or traceable. You do not know which skill produced which message. There is no visual indicator in the chat showing which skill was used.

-
One session at a time. You can only work on one subject at a time. There is no way to run multiple sessions in parallel and switch between them.

-
Adding or changing skills requires manual intervention. You cannot add or modify skills through an interface; it requires direct work on the system.

1.3 What Hermes-Kore is supposed to achieve

The goal of Hermes-Kore is to introduce a dedicated graphical interface - a Web Progressive App (WPA) - that solves these frictions while keeping Hermes as the central orchestrator.

Specifically, Hermes-Kore should:

- Add a WPA interface alongside Telegram, so you have a visual screen to manage the system.

- Keep Hermes as the brain. Hermes remains the orchestrator that delegates work, manages skills, talks to AI models, and executes tasks. The WPA is a view on top of Hermes, not a replacement.

- Make the system operable from a dashboard where you can see workers, tasks, lanes, monitoring, and skills in one place.

1.4 Who this is for

The primary user is you, K Ray Nobun. You manage your own projects through this system. Hermes-Kore is not initially built for teams or organizations, though its structure (administrator, supervisors, workers, cronjobbers) is inspired by enterprise organization.

1.5 What success looks like

When Hermes-Kore is working, you should be able to:

- Open a dashboard and see what every sub-agent is doing right now

- Know which tasks are completed, in progress, or blocked

- Monitor your VPS resources in real time

- Give Hermes a goal and have it break it down, execute it, and report back - without you watching every step

- Receive notifications only when something is finished or when a decision truly needs you

2. Separation of Communication vs Reflection

2.1 The problem as you described it

Right now, when you talk to Hermes on Telegram, everything arrives in one stream. A message might be:

- An answer to your instruction (Flux A)

- A deliverable or file produced by a task (Flux A)

- A validation or confirmation (Flux A)

- A blocking error that needs your attention (Flux A)

- Internal reasoning (Flux B)

- Self-correction notes (Flux B)

- Debug logs (Flux B)

You cannot tell at a glance which messages are meant for you and which are Hermes thinking to itself. The important signals get buried in the internal noise.

2.2 What the requirement asks for

Two clearly separated flows:

Flux A - Communication to you:

- Responses to your instructions

- Deliverables and produced files

- Validations and confirmations

- Errors that block progress and need your decision

Flux B - Internal reflection:

- Hermes's internal reasoning

- Auto-correction notes

- Debug logs

- Self-examination of whether a task was done well

Flux B should be hidden by default. You should see only Flux A unless you choose to look at Flux B through a "details mode" or a dedicated side panel.

2.3 What already exists in the codebases

Hermes side:

Hermes does have a concept of thinking. It maintains sessions with context, and it produces reasoning traces as it works. On some surfaces (CLI, desktop) the thinking feed is more visible; on Telegram it is more compressed. There is no clean switch between "show me only results" and "show me everything" on Telegram.

Kore-OS side:

Kore-OS has a PWA control center with:

- A thinking feed in real time (WebSocket on port 1020)

- An intervention channel (WebSocket on port 1035) where an operator can step in mid-run

- Mission tabs (Missions, Logs, Artifacts, Stats)

- Real-time push notifications for critical states

These two WebSocket channels already separate live reasoning from control, but they are inside the Kore PWA, not exposed through Telegram.

2.4 How Hermes-Kore should implement this

The WPA interface should have:

- A main view showing only Flux A - what you need to see and act on

- A toggle or side panel for Flux B - the internal thinking, available on demand

- Clear visual distinction so you never confuse a deliverable with a debug log

Telegram should continue to carry Flux A messages (results, completions, blockers) so you are not forced to open the WPA for everyday operation.

2.5 What changes for you day to day

- You open Telegram and see only what matters: answers, files, confirmations, errors needing you.

- You open the WPA when you want to dig into how Hermes reasoned, what it corrected, or what it was thinking during a long task.

- You are never forced to read internal monologue to find the result of your instruction.

3. Automatic API Model Switching

3.1 The problem as you described it

Hermes can talk to several AI model providers: Gemini, NVIDIA, OpenRouter, Tencent, and others. Right now, when one model fails - for example because of a 429 rate limit, a 402 insufficient balance, or a 401 invalid key - the switch to the next model does not happen reliably or automatically. You may have to intervene manually to get Hermes working again.

3.2 What the requirement asks for

An automatic fallback mechanism between AI models with no manual intervention:

- If a model fails, the system switches to the next one in a configured backup chain.

- The switch is transparent to you - you should not have to do anything.

- The backup chain is configurable: you can set the order, the models, and the API keys used.

3.3 What already exists in the codebases

Hermes side:

Hermes already has a working fallback chain, measured in production:

- Primary provider: NVIDIA (base URL: integrate.api.nvidia.com/v1)

- Fallback provider: OpenRouter, model anthropic/claude-sonnet-4.6

- Two NVIDIA API keys in a credential pool with automatic rotation between them

- Cooldown per HTTP status code (429, 503, quota errors)

- 52 credential rotation events observed in a single day (25/10), showing the mechanism absorbed provider failures without human intervention

- The delegation model was changed from glm-5.3-flash (which timed out 5/5 times) to glm-5.3, and the system continued working

Kore-OS side:

Kore-OS has an api_guard middleware that:

- Detects 429/503/quota errors

- Rotates keys within a single provider with a 60-second cooldown

- Loads the key pool encrypted through an AES-256-GCM vault

- Has cross-provider support only for zhipufigemini on the LLM core side

- The cross-provider transverse fallback remains unimplemented - it is mono-provider in practice

3.4 How Hermes-Kore should implement this

The fallback chain should be:

- Try the primary model (e.g., NVIDIA with key 1)

- If it fails with 429/401/402/503, rotate to the next key in the pool for the same provider

- If all keys for that provider are exhausted, switch to the next provider in the chain (e.g., OpenRouter)

- If all providers fail, present the problem to you with potential solutions

The chain should be configurable through an interface where you can set the order of providers, which models to use, and which API keys belong to each.

3.5 What changes for you day to day

- When one AI provider has an outage or hits a rate limit, Hermes keeps working automatically.

- You do not need to restart or reconfigure anything.

- You can see which model is active and adjust the backup chain if you want.

4. Dedicated Skills Directory and Traceability

4.1 The problem as you described it

The skills Hermes uses are not explicitly listed, and you do not know which skill generated which message. Skills are used without any visual indicator in the chat, so you cannot trace a result back to the skill that produced it.

4.2 What the requirement asks for

A dedicated skills directory (centralized catalog) where:

- Every message generated by a skill is traceable back to that skill

- Each message bubble in the WPA interface has a visual indicator - a colored border or a small seal in a corner - showing which skill was used

- Hovering over a message shows the skill name (for example: "skill of cron job," "skill of deployment," "skill of writing an article")

- A color legend is available somewhere in the interface explaining what each color means

4.3 What already exists in the codebases

Hermes side:

Hermes has a functioning skills system:

- 23 skill directories on the VPS as of the measurement date

- Skills auto-learn from sessions: lessons learned are written into skill markdown files

- A skill hub (hermes-agent skill) with 17 references

- Skills are loaded by session and can be added hot (no rebuild needed)

- Two hard invariants: never break prompt caching, and secrets go in .env not in config.yaml

- Skills extend Hermes's capabilities without changing code - a new skill is just a markdown file

Kore-OS side:

Kore-OS has a typed tool system:

- 16 ToolAction types, each mapped to a trait (CONCEPTION, REALIZATION, EVALUATION)

- A TRAIT_ACTION_MAP registry that prevents a tool from acting outside its trait

- 24 tests covering the trait system

- Adding a tool requires modifying the registry, recompiling, and retesting - high cost, guaranteed consistency

4.4 How Hermes-Kore should implement this

A centralized skill catalog should:

- List all skills with description, status (active/inactive), creation date, and last use date

- Allow you to add new skills and modify existing ones from the WPA interface

- Tag every message with its source skill using a visual indicator (colored border, seal, or badge)

- Show the skill name on hover

- Provide a color legend explaining each skill color

- Trace messages back to skills so you can always answer: "Which skill produced this?"

4.5 What changes for you day to day

- You can see at a glance which skill handled each message.

- You can add or change skills from the interface without touching code.

- You have a catalog showing what skills exist, which are active, and when each was last used.

- You can verify that the right skill is being used for the right kind of work.

5. Multiple Sessions and Alternation

5.1 The problem as you described it

Hermes can only handle one session at a time. This limits your ability to work on several subjects in parallel. If you are in the middle of a task and want to start something else, you have to finish or pause the current subject first.

5.2 What the requirement asks for

- You must be able to launch several sessions at the same time.

- You must be able to switch between sessions from the WPA interface (session selector, tabs, or a side list).

- Each session keeps its own context and its own history. Switching sessions does not mix their contents.

5.3 What already exists in the codebases

Hermes side:

Hermes currently runs one conversation at a time through its gateway. It has persistent memory (user profile and

notes store) that survives across conversations, but there is no multi-session interface within a single interaction. You can start new conversations, but they are not organized as switchable sessions inside one view.

Kore-OS side:

Kore-OS has a mission-based model where each mission is independent and can be tracked separately. The PWA has mission tabs and a mission list, so you can see multiple missions at once. However, this is mission-level, not session-level with isolated context.

5.4 How Hermes-Kore should implement this

The WPA should provide:

- A session selector or tab bar where you can see all open sessions

- The ability to create a new session at any time

- Clicking a session switches the main view to that session's context and history

- Each session maintains its own task list, worker assignments, lane membership, and conversation history

- If technical constraints prevent true parallel sessions, Lanes (Section 9) must at least provide context isolation between tracks of work

5.5 What changes for you day to day

- You can work on several projects at once without losing your place in any of them.

- You switch between sessions with a click, and each one shows exactly where you left off.

- Context from one project does not leak into another.

6. Real-Time VPS Monitoring

6.1 The problem as you described it

Your VPS resource data (RAM, CPU, SWAP, disk) is not displayed in real time in the interface. Cron deliveries give you snapshots, but that is not continuous monitoring. You cannot see current load at a glance.

6.2 What the requirement asks for

A program running on the VPS that provides resource data directly - no AI call, no LLM intermediary. The WPA should display in real time:

- RAM usage percentage

- CPU usage percentage

- SWAP usage percentage

- Disk space used vs total

- System load (1 minute, 5 minutes, 15 minutes)

The display should refresh automatically (polling or WebSocket).

6.3 What already exists in the codebases

Hermes side:

Hermes runs on the VPS and can execute terminal commands, but there is no dedicated monitoring program streaming VPS metrics to an interface. You would need to ask Hermes to run a command to get a snapshot, which is not real time.

Kore-OS side:

Kore-OS has a system monitor (system_monitor.ts) that tracks emergency thresholds (RAM 85%, CPU 180 seconds, FD 1024) and can kill processes that exceed them. It also has push notifications triggered by critical thresholds. However, this is focused on safety thresholds, not on providing a real-time metrics display for you.

6.4 How Hermes-Kore should implement this

A lightweight program on the VPS should:

- Collect RAM, CPU, SWAP, disk, and load data directly from the operating system (no AI involved)

- Expose this data through a simple endpoint or WebSocket

- The WPA subscribes to this feed and displays the numbers updating automatically

- Refresh happens without you needing to reload the page

6.5 What changes for you day to day

- You open the WPA and see your VPS health at a glance: RAM, CPU, SWAP, disk, load.

- The numbers update continuously without you refreshing.

- You know immediately if the VPS is under pressure, without having to run commands.

7. Dynamic Skills Management

7.1 The problem as you described it

Adding or modifying a skill currently requires manual intervention on the system. You cannot simply add a new capability or change an existing one through an interface.

7.2 What the requirement asks for

- You can add new skills from the WPA interface.

- You can modify existing skills from the WPA interface.

- The skill catalog is viewable with description, status (active/inactive), creation date, and last use date.

7.3 What already exists in the codebases

Hermes side:

Hermes already has a working skills system:

- Skills are stored as markdown files in directories (23 directories measured)

- New skills can be added by placing a markdown file in the skills directory - no code change, no rebuild

- Skills are loaded by session, so a new skill becomes available immediately

- Skills auto-learn: after a session, lessons are written into skill files

- The skill hub (hermes-agent skill) manages 17 references and provides the orchestration procedure

- Two invariants are enforced: never break prompt caching, and secrets stay in .env not in config.yaml

Kore-OS side:

Kore-OS uses a typed tool system (16 ToolActions with traits) where adding a capability requires modifying the registry, recompiling, and retesting. This is the opposite of dynamic: it is rigid and requires a build cycle.

7.4 How Hermes-Kore should implement this

The WPA should provide:

- A skill catalog page listing every skill with its description, status, creation date, and last use date

- A form to add a new skill (name, description, what it does)

- A form to edit an existing skill (change description, activate/deactivate)

- The underlying mechanism uses Hermes's hot-reloadable markdown skill files, so changes take effect immediately without restart

7.5 What changes for you day to day

- You can give Hermes a new capability just by describing it in the interface.

- You can turn skills on or off without technical work.

- You can see which skills exist and when each was last used.

- No code editing or system restart is needed to add or change a skill.

8. Sub-Agent Roles

8.1 The problem as you described it

Sub-agents do not have clearly defined or displayed roles. You cannot tell at a glance whether a sub-agent is an administrator, a supervisor, a worker, a cronjobber, or a researcher. Everyone looks the same.

8.2 What the requirement asks for

Five explicit roles, defined and assigned to each sub-agent:

- Admin - administration of the system, configuration, access rights.

- Supervisor / Manager - supervision of other workers, task assignment, quality control.

- Worker / Implementer - concrete execution of modifications (code, files, deployment).

- Cron Jobber - execution of recurring scheduled tasks.

- Researcher - information search, documentation reading, answering factual questions.

Each worker must display its current role in the interface.

8.3 What already exists in the codebases

Hermes side:

Hermes does not currently assign or display roles to its sub-agents. Workers are created as sub-agents with a subset of context, but they do not have an explicit role label shown to you.

Kore-OS side:

Kore-OS has a trait system with three traits (CONCEPTION, REALIZATION, EVALUATION) that map to actions. These are closer to capabilities than to roles, and they are internal - not displayed to you as worker roles.

8.4 How Hermes-Kore should implement this

Each sub-agent should be assigned one of the five roles at creation:

- Admin: The main orchestrator. Talks to you, understands goals, delegates tasks, decides which model to use.

- Supervisor: Checks worker output quality, assigns daily tasks, requests corrections, reports progress to Admin.

- Worker: Executes concrete tasks - writing, coding, research, data entry.

- Cron Jobber: Runs scheduled routine tasks automatically.

- Researcher: Searches information, reads documentation, answers factual questions.

The WPA dashboard should display each worker's role next to its name, so you can see at a glance who is what.

8.5 What changes for you day to day

- You know what kind of sub-agent you are looking at: admin, supervisor, worker, cronjobber, or researcher.

- You know who is responsible for what: admin for delegation, supervisor for quality, workers for execution, cronjobbers for schedules, researchers for information.

- The enterprise structure becomes visible instead of hidden.

9. Lanes (Parallel Work Tracks)

9.1 The problem as you described it

When several tasks run in parallel, they risk mixing their contexts and creating confusion. You cannot tell which task belongs to which project, and information from one task leaks into another.

9.2 What the requirement asks for

Introduce the concept of a Lane:

- A Lane is a dedicated track of work, isolated contextually.

- A Lane can execute several tasks without their contexts mixing.

- A Lane can have several workers assigned to it (for example, one implementer plus one tester).

- If multiple sessions are not technically possible, Lanes must at least provide this context isolation.

- Each Lane has an ID, a name, a status, and a list of workers assigned to it.

9.3 What already exists in the codebases

Hermes side:

Hermes does not have a Lane concept. Workers are delegated individually without being grouped into context-isolated tracks.

Kore-OS side:

Kore-OS organizes work into missions, each with its own state. Missions are separate, but there is no explicit "Lane" construct that groups multiple tasks and workers under one context-isolated track.

9.4 How Hermes-Kore should implement this

The WPA should provide:

- A way to create Lanes with a name and description

- The ability to assign workers to a Lane

- A view showing all tasks running within a Lane

- Context isolation: tasks in Lane A cannot read files, notes, or context from Lane B unless explicitly shared

- Each Lane shows its status (active, paused, closed) and its assigned workers

9.5 What changes for you day to day

- You can group related tasks under one Lane (for example, "Client X Campaign" or "Website Redesign").

- Tasks in different Lanes do not interfere with each other.

- You can see which workers are assigned to which Lane.

- You can pause or close a Lane without affecting other Lanes.

10. Dashboard Workers and Purpose Narration

10.1 The problem as you described it

You do not know who is doing what, and technical details (which .js file was changed, which npm package errored) do not interest you as much as the business outcome. You want to see what each worker accomplishes in plain terms.

10.2 What the requirement asks for

A main dashboard showing all active workers in real time. For each worker:

- Name / ID

- Role (admin, supervisor, worker, cron, researcher)

- Lane assignment

- Status (inactive, in progress, blocked, finished)

A purpose narration for each worker - a description of what the worker accomplishes in business terms, not technical terms:

- Good example: "This worker makes it possible to fill forms with options X1, X2..., so the user will act exactly as follows: ..."

- Bad example (to avoid): "This worker modifies the file form.js and encounters an npm package error."

The dashboard must be in the main area of the WPA interface.

10.3 What already exists in the codebases

Hermes side:

Hermes has a kanban board configured, and it tracks delegated workers in state.json. However, there is no visual dashboard showing all workers with their roles, lanes, and statuses. Workers are not labeled by role, and their purpose is not narrated in business terms.

Kore-OS side:

Kore-OS has a PWA control center with mission tabs (Missions, Logs, Artifacts, Stats) and a thinking feed. It shows mission status, but not a worker dashboard with roles and purpose narration.

10.4 How Hermes-Kore should implement this

The WPA dashboard should:

- List every active worker in a table or card view

- Show for each worker: name, role, lane, status

- Show a purpose narration written in business terms for each worker

- Update in real time as workers change status

- Be the main view of the WPA, not hidden in a submenu

10.5 What changes for you day to day

- You open the dashboard and see everyone at once: who is working, who is blocked, who is finished.

- You read what each worker is accomplishing in plain language, not technical jargon.

- You can spot a blocked worker immediately and decide what to do.

- You never have to dig into file names or error messages to understand what is happening.

11. Provider Quotas and Color Coding by Model

11.1 The problem as you described it

You do not know which AI model is active on which sub-agent, nor how much quota remains available. You cannot tell if you are running out of credits on one provider while another sits unused.

11.2 What the requirement asks for

- Display of provider quota usage: per minute and per day.

- Display of the active AI model on each sub-agent, regardless of type.

- A color code per model: each AI model gets a color (border or badge).

- A legend explaining which color corresponds to which model.

- Example: Gemini = blue, NVIDIA = green, OpenRouter = orange, Tencent = red.

11.3 What already exists in the codebases

Hermes side:

Hermes has a credential pool with two NVIDIA API keys and automatic rotation. It logs 52 credential rotation events in one day. It has a fallback to OpenRouter/Claude-Sonnet-4.6. However, there is no display of quota usage (per minute, per day) and no color coding by model shown to you.

Kore-OS side:

Kore-OS has an api_guard that rotates keys within a provider and tracks key pool state, but it does not show you quota per minute/per day or color-code models.

11.4 How Hermes-Kore should implement this

The WPA should:

- Show a quota panel: for each provider, how many calls per minute and per day remain

- Show which model is active on each sub-agent (next to the worker name or in the worker detail view)

- Assign a color to each model (for example: NVIDIA = green, OpenRouter = orange, Anthropic = blue)

- Display that color as a badge or border on the worker or in a model indicator

- Provide a legend table or key explaining which color maps to which model

11.5 What changes for you day to day

- You can see at a glance which model each worker is using.

- You know how much quota is left on each provider, per minute and per day.

- You can spot if one provider is being overused while another is idle.

- The color coding lets you identify models quickly without reading names.

12. 100% Automatic Background Tasks

12.1 The problem as you described it

When a background task is defined, the system asks after execution whether it should be resumed. This breaks automation - you want background tasks to run without being asked.

12.2 What the requirement asks for

- A special category of tasks that run 100% in the background, pre-established.

- When such a task is given, it executes automatically without asking for confirmation or resume.

- You are not asked afterward: "Should we resume the background task?" - No, it just runs.

- These tasks are listed in a dedicated section of the interface.

12.3 What already exists in the codebases

Hermes side:

Hermes has a dedicated cron scheduler process (separate from the gateway) that runs on a 10-minute cycle. It sends status messages regularly. However, no workers are currently active, and background tasks may require

confirmation after running.

Kore-OS side:

Kore-OS has a mission engine with a scheduler, checkpoints, and multi-day persistence. Missions can be started, paused, resumed, and cancelled through API routes. The engine supports long-running missions with checkpoints every 5 iterations.

12.4 How Hermes-Kore should implement this

The WPA should provide:

- A dedicated section for background tasks (separate from regular tasks)

- A way to mark a task as "100% automatic background"

- Automatic execution without confirmation prompts after the task runs

- No resume question - once started, it completes on its own

- A list view of all background tasks with their schedule and status

12.5 What changes for you day to day

- You define a background task once and it runs on its schedule without you being asked.

- You never see a "should we resume this?" prompt for background tasks.

- All background tasks are listed in one place so you can see what is running automatically.

13. Instructions View and Status by Lane

13.1 The problem as you described it

You do not have a consolidated view of the instructions you have given, their status, or the history of exchanges by Lane. You cannot see at a glance what you asked for, what is done, what is blocked, or where each instruction stands.

13.2 What the requirement asks for

A global view of all instructions:

- A list of every instruction given to Hermes/Hermes-Kore

- For each instruction:

- The instruction text

- Status (pending, in progress, completed, blocked, cancelled)

- Associated Lane

- Latest messages from that instruction's Lane

- Creation date and last update date

- Ability to filter by status, by Lane, and by date

13.3 What already exists in the codebases

Hermes side:

Hermes has a state database (state.db) with sessions and kanban data, but there is no consolidated view of all instructions with their status and Lane association that you can browse.

Kore-OS side:

Kore-OS tracks missions with start/pause/resume/cancel/priority/budget routes and has an experience_snapshots table linked to missions. However, there is no instruction-by-Lane view designed for you to browse all your instructions.

13.4 How Hermes-Kore should implement this

The WPA should provide:

- A dedicated page listing every instruction you have given

- For each instruction: full text, current status, associated Lane, latest messages, creation date, last update date

- Filter controls: by status, by Lane, by date range

- Clicking an instruction shows its full history and the Lane it belongs to

13.5 What changes for you day to day

- You can see every instruction you have ever given, in one list.

- You know the status of each one without asking.

- You can filter to see only blocked tasks, or only tasks in a specific Lane.

- You can review the history of any instruction to see how it progressed.

14. Delivery Modalities for Worker "Pythagor"

14.1 What the requirement asks for

When the worker "Pythagor" is assigned to the Hermes-Kore project, he must:

- Read this specification in full.

- Produce a phased implementation plan (MVP -> complete version).

- Keep an architecture decision journal.

- Validate each feature with you (the commanditaire) before moving to the next.

- Never include API keys, tokens, or secrets in plain text in any deliverable - replace them with [REDACTED].

14.2 What this means for the project

Pythagor is the worker assigned to build Hermes-Kore. His work must be:

- Transparent: you see the plan before he starts.

- Validated: you approve each feature before he builds the next one.

- Secure: no secrets ever appear in deliverables.

- Documented: he keeps a journal of architectural decisions so you know why choices were made.

Section 14 complete. All specification sections documented.

15. The Worker Class Model (Revolutionary Concept)

15.1 What is a worker class?

You described a fundamental concept: you create classes of workers. A class is not a worker itself - it is a template or mold. For example, you can define a "motion design" worker class. Later, whenever you need motion design work, you make an instance of that class. You can make many instances, and each instance is called a worker.

15.2 What a class defines

Each worker class you create specifies:

- The set of skills it has access to. You provide these skills when defining the class. These are worker skills - they belong to the class and are available to every worker instantiated from it.

- A system prompt. Originally from the class. Every worker of the same class starts with the same base system prompt, which describes the worker's ability and identity.

- Reference documents. A group of documents that provide information for few-shot prompting or reasoning - i.e., how the worker should reason when it thinks.

- Personalization placeholders. Optional placeholders in the system prompt so that one worker of a class can be specialized on one thing and another worker of the same class on a different thing. This avoids hallucination and produces more precise results.

- The AI model it should use. Potentially different per class.

15.3 Class instantiation

When you need a worker:

- You select a worker class.

- You instantiate it - create a worker instance from the class.

- The worker inherits the class skills, base system prompt, documents, and model.

- If placeholders exist, you fill them for this specific instance.

- The worker can now use everything it has: skills, prompt, documents, model.

15.4 Example

You define a "motion design" worker class with:

- Skills: keyframe generation, video preview rendering, After Effects scripting

- System prompt: "You are a motion designer specialized in {software}. You reason step by step before rendering."

- Placeholder: {software} -> filled as "After Effects" for one worker, "Blender" for another

- Documents: brand guide, style references, animation principles

- Model: a video-capable AI model

You then instantiate two workers from this class: one specialized in After Effects, one in Blender. Both are the same class but produce different, precise results.

15.5 What changes for you day to day

- You define worker types once (classes) and reuse them whenever needed.

- Each worker knows exactly what it can do, what it should sound like, and which model to use.

- You can create many workers from the same class for parallel work.

- Placeholders let you specialize workers without creating new classes.

16. Skills System: Two-Tier Architecture

16.1 Worker skills

Worker skills belong to a worker class and are available to every instance of that class. You give the worker class a set of skills; it has access to them. Examples: web search, file write, email sending, video rendering, terminal execution.

16.2 Context skills

Context skills are different from worker skills. They are crafted by a sub-agent when, during work, it learns something specific to the project context - something more related to the project than to the worker's own nature. For example, a worker writing a marketing campaign might learn the client's tone of voice, product positioning, and forbidden claims. That knowledge is packaged as a context skill.

Properties of context skills:

- Created by a sub-agent during work

- Project-specific, not worker-specific

- Usable by anyone, not locked to the creator

- Assigned by the Administrator or Supervisor when a sub-agent must work in that specific context

- Stored in a context skill library, separate from the worker class skill library

16.3 Skill lifecycle

- Definition: You define worker skills when creating a worker class.

- Assignment: Context skills are created by sub-agents and assigned by Admin/Supervisor.

- Usage: A worker uses its class skills plus any context skills assigned for the current task.

- Revision: Skills can be added or modified. You manage worker skills; Admin/Supervisor manage context skill assignments.

- Catalog: A centralized skill directory lists all skills with description, status, creation date, and last use date.

16.4 What changes for you day to day

- You provide the skills each worker class needs.

- Workers learn project-specific things and package them as context skills anyone can use.

- You always know which skills exist and which are active.

17. Prompts, Identity, and AI Models

17.1 System prompt per class

Every worker class has a system prompt describing the worker's ability and identity. All instances of the class start with the same base system prompt.

17.2 Creation prompt

When a worker is created, the Administrator gives it a prompt on its creation. The worker lives with this prompt every single time it works. This is separate from the class system prompt - it is the worker's standing instruction set for its entire lifetime.

17.3 Placeholders and personalization

The system prompt may contain placeholders. When instantiating a worker, you fill placeholders to specialize the worker. One class can spawn precise variants without hallucination.

17.4 AI model assignment

Different sub-agents use different AI models. Each worker class specifies the model it should use. The Administrator can also decide which model each sub-agent uses. Models are selected based on fit for the work, not uniformity.

17.5 Prompt caching

Prompts should be designed to allow prompt caching. The system must not break prompt caching. Skills index and prompt content should be managed so token cost per turn remains predictable.

18. Internal Logic and Workflows

18.1 Goal intake and decomposition

When you give KORE-OS a goal:

- A specific sub-agent asks you clarifying questions.

- The system presents a plan to you for approval.

- Once approved, a Worker executes the task.

- The Worker reports the result.

The flow is: Clarify -> Approve -> Execute -> Report. You approve the plan before execution begins.

18.2 Delegation logic

The Administrator chooses which sub-agent gets a task by matching the task type to the sub-agent's specialty. It can reassign a task mid-way automatically if things are not going well.

18.3 Failure handling and escalation

When a sub-agent cannot complete a task:

- It tries a different approach automatically.

- If it still fails, it presents:

- What the problem is

- What the potential solutions are

- What each solution should imply (consequences, cost, time)

- You or the Administrator decides next steps.

The system does not silently fail.

18.4 Learning and context skills

When doing work, a sub-agent may learn something specific to the project context. It crafts a context skill from that learning. The Administrator or Supervisor assigns it to the relevant sub-agent when that sub-agent must work in that context.

Sub-agents remember only important lessons, not everything.

18.5 Priority and scheduling

Task priority is determined in this order:

- You set priorities manually at any time.

- Deadlines and urgency.

- The Administrator's judgment of what matters most.

You can override priorities at any time.

19. Communication and Notification Chain

19.1 Worker -> Supervisor -> Admin -> You

The standard notification chain is:

- Worker notifies its Supervisor when a task is done or blocked.

- Supervisor quality-checks and notifies the Administrator.

- Administrator notifies you only when:

- The task you gave is completed, OR

- There is something really important the Administrator cannot decide on its own.

You are not notified of every intermediate step. The chain filters noise.

19.2 Supervisor strictness

Supervisors are extremely critical. Before passing any worker output upward:

- They verify the output is excellent.

- They reject unwanted details.

- They ask the worker to correct specific points.

- They are realistic and grounded; they do not overpromise.

If a Supervisor reports a result that is not what was demanded, the Supervisor is accountable.

19.3 Correction loops

When quality is insufficient:

- Supervisor returns the work to the Worker with specific correction points.

- Worker revises using its skills and context.

- Supervisor re-checks.

- Only when excellent does the work move to the Administrator.

20. Reliability, Recovery, and Limits

20.1 Crash recovery

If a sub-agent stops responding or crashes, the Administrator restarts the task from the last saved checkpoint. Work is not lost - progress is preserved through checkpoints and snapshots.

20.2 Pause/resume

The system can pause a long task and resume it later without losing progress. This applies to very long tasks (hours or days).

20.3 Limits

Limits on how much a sub-agent can do (time, cost, steps) are loose guidelines. The system does not stop good work arbitrarily. You can adjust or remove limits.

20.4 Checkpoints and snapshots

The mission engine checkpoints work periodically, takes full snapshots on suspension, and can restore from snapshot at resume with budget rehydrated.

21. Integration with Hermes and Kore-OS

21.1 What Hermes brings

Hermes is the mature AI agent harness running on Telegram, CLI, desktop, web dashboard, and API. It provides:

- Multi-platform interaction

- Credential pool with automatic model fallback (429/401/402 handling)

- Skills auto-learning

- Cron scheduler

- Skills directory (23+ directories)

- Persistent memory

Hermes has no durable sandbox, no multi-day missions, no traits, and no git-state rollback.

21.2 What Kore-OS brings

Kore-OS is a TypeScript/Node minimal OS with:

- Persistent mission loop with micro-steps

- Checkpoints every 5 iterations

- Multi-day snapshots

- Crash recovery

- Git-state at /etc/elo-state with rollback

- AES-256-GCM vault for secrets

- Sandbox execution with security hardening

- Push notifications

- PWA Control Center

Kore-OS has only one interface (PWA), no proven end-to-end runtime on the VPS, and no multi-provider LLM fallback chain.

21.3 Fusion architecture

The recommended fusion is: Hermes shell + Kore-OS brain via MCP (Model Context Protocol).

- Hermes remains the operator shell: Telegram, CLI, desktop, skills, model pool, fallback chain.

- Kore-OS remains the mission engine: persistent loops, checkpoints, snapshots, sandbox, git-state, vault.

- Hermes calls Kore-OS tools through MCP.

- No codebase is rewritten - Hermes adds an MCP server block in its config.

Section 21 complete. All recursive documentation sections finished.